

Lecture 9: From C to the ISA

CMPE I20: Computer Organization & Architecture

Olivia Weng

Logistics

- HW1 is due tonight (!) at 11:59pm PT
- HW1 Quiz next Tuesday [in class](#)
 - 15min

What does a C program compile to?

What does a C program compile to?

```
#include <stdio.h>

int main() {
    printf("Hello world!");
}
```

What does a C program compile to?

- Assembly!

```
#include <stdio.h>

int main() {
    printf("Hello world!");
}
```

What does a C program compile to?

- Assembly!

```
#include <stdio.h>

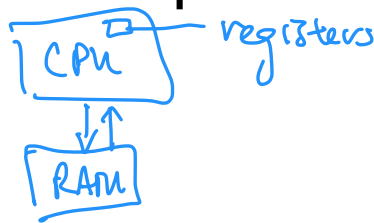
int main() {
    printf("Hello world!");
}
```

x86

```
.LC0:
    .string "Hello world!"
"main":
    push    rbp
    mov     rbp, rsp
    mov     edi, OFFSET
FLAT:.LC0
    mov     eax, 0
    call    "printf"
    mov     eax, 0
    pop     rbp
    ret
```

What does a C program compile to?

- Assembly!



```
#include <stdio.h>
```

```
int main() {
    printf("Hello world!");
}
```

x86

```
.LC0:
.string "Hello world!"
"main":
    push    rbp
    mov     rbp, rsp
    mov     edi, OFFSET FLAT:.LC0
    mov     eax, 0
    call    "printf"
    mov     eax, 0
    pop     rbp
    ret
```

ARM

```
main:
    stp     x29, x30, [sp, #-16]!
    mov     x29, sp
    adrp    x0, .L.str
    add     x0, x0, :lo12:.L.str
    bl      printf
    mov     w0, wzr
    ldp     x29, x30, [sp], #16
    ret

.L.str:
.asciz    "Hello world!"
```

RISC-V

```
.LC0:
.string "Hello world!"
main:
    addi    sp, sp, -16
    sd      ra, 8(sp)
    sd      s0, 0(sp)
    addi    s0, sp, 16
    lla     a0, .LC0
    call    printf@plt
    li      a5, 0
    mv      a0, a5
    ld      ra, 8(sp)
    ld      s0, 0(sp)
    addi    sp, sp, 16
    jr      ra
```

we will use this!

Where does assembly come from?

- The assembly language is defined by an [instruction set architecture \(ISA\)](#)

Where does assembly come from?

- The assembly language is defined by an **instruction set architecture (ISA)**
- Recall from Lecture 1:

Computer architecture = **Instruction set architecture** + **Machine organization**

↳ how we talk
to the hardware

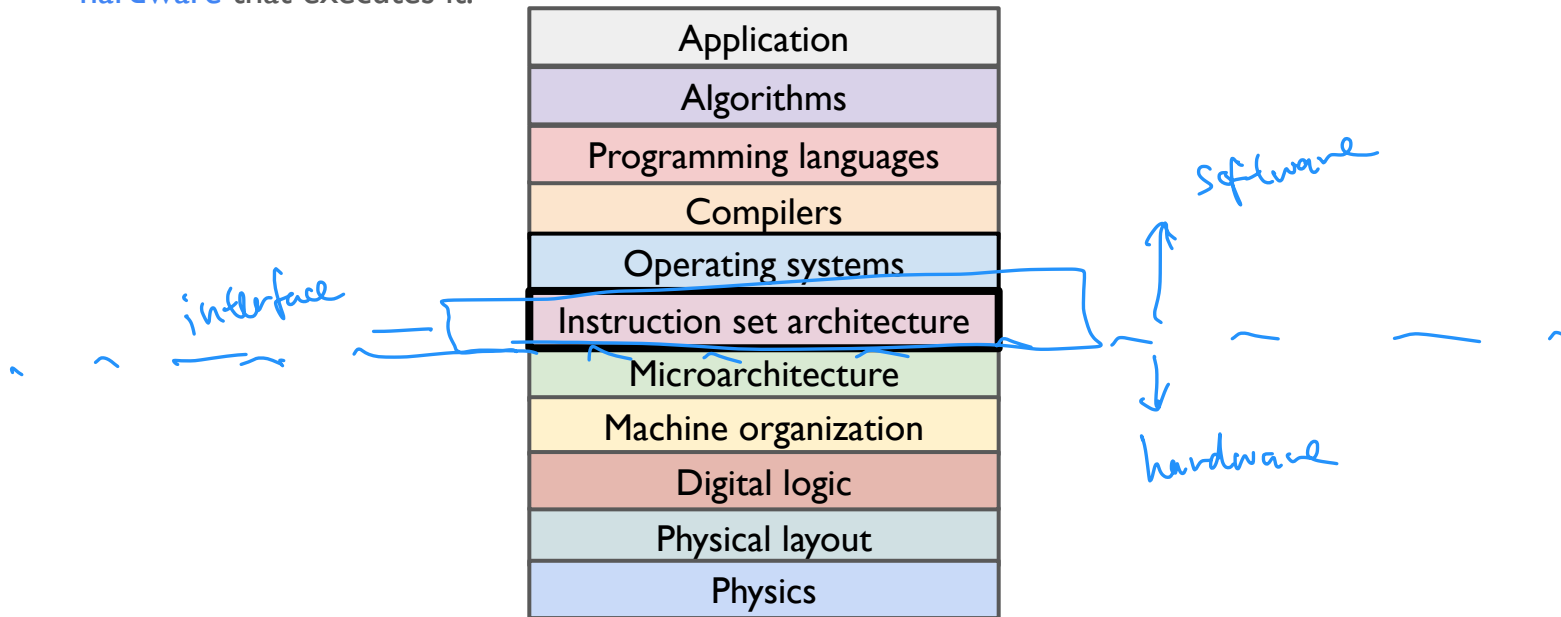
↑ how the hardware
actually executes

What is an Instruction Set Architecture (ISA)?

- An ISA describes an abstraction of a computational machine
 - Describes the “the agreed-upon interface between all the software that runs on the machine and the hardware that executes it.”

What is an Instruction Set Architecture (ISA)?

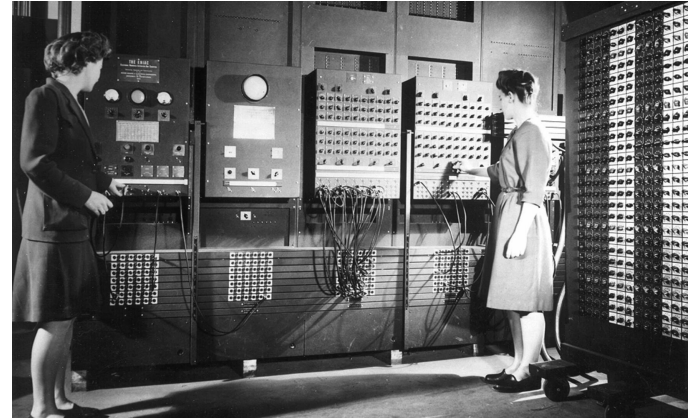
- An ISA describes an **abstraction** of a **computational machine**
 - Describes the “the agreed-upon **interface** between all the **software** that runs on the machine and the **hardware** that executes it.”



Why have an **Instruction Set Architecture (ISA)**?

In the beginning...

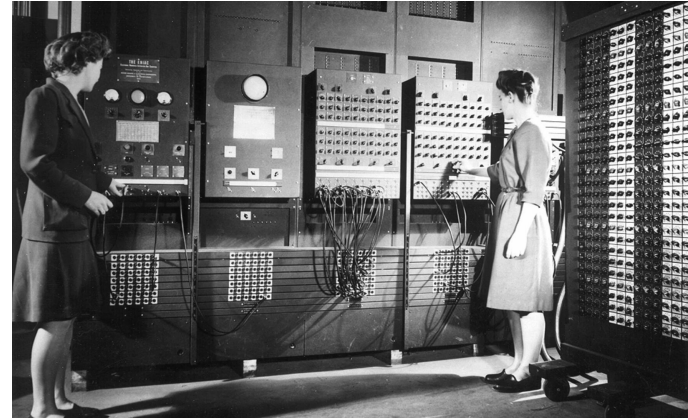
- The first computers used **physical configuration** as their “program”
- Technicians would **physically reconfigure** the machine.
 - Slow!



Why have an **Instruction Set Architecture (ISA)**?

In the beginning...

- The first computers used **physical configuration** as their “program”
- Technicians would **physically reconfigure** the machine.
 - Slow!
- Didn't let us **use the computer as a tool** for programming



Why have an **Instruction Set Architecture (ISA)**?

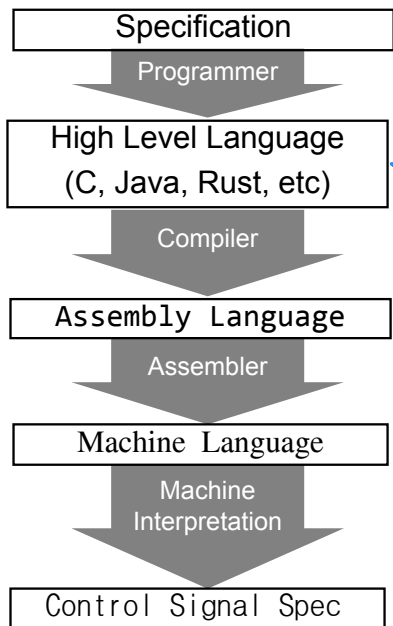
- To use the computer as a tool for programming!

Why have an **Instruction Set Architecture (ISA)**?

- To use the computer as a tool for programming!
- Idea: The **stored-program computer** where the computer comprises
 - A **central processing unit (CPU)** that
 - Executes instructions
 - Holds a small number of values in a “register file”
 - A **memory** that holds data and instructions

Why have an Instruction Set Architecture (ISA)?

- Computers **do not** speak English or any programming language (C, Java, etc.)



"Swap two array elements."

```
int temp = array[index];  
array[index] = array[index + 1];  
array[index + 1] = temp;
```

```
lw    $15, 0($2)  
lw    $16, 4($2)  
sw    $16, 0($2)  
sw    $15, 4($2)
```

loads
stores

```
10001100011000100000000000000000  
10001100111100100000000000000100  
10101100111100100000000000000000  
10101100011000100000000000000100
```

machine code

```
ALUOP[0:3] <= InstReg[9:11] &  
MASK
```

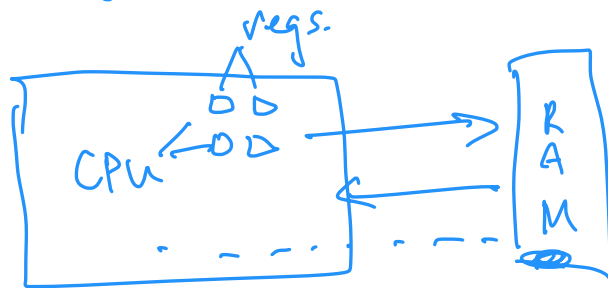
Hardware
Description Language
HDL

The Instruction Set Architecture (ISA)

- The ISA is the **interface** between software and hardware

The Instruction Set Architecture (ISA)

- The ISA is the **interface** between software and hardware
- The part of the architecture that is **visible** to the **programmer**
 - available **instructions** (“opcodes”)
 - number and types of **registers** — *ex: zero register*
 - instruction **formats**
 - **storage** access, **addressing** modes
 - exceptional conditions

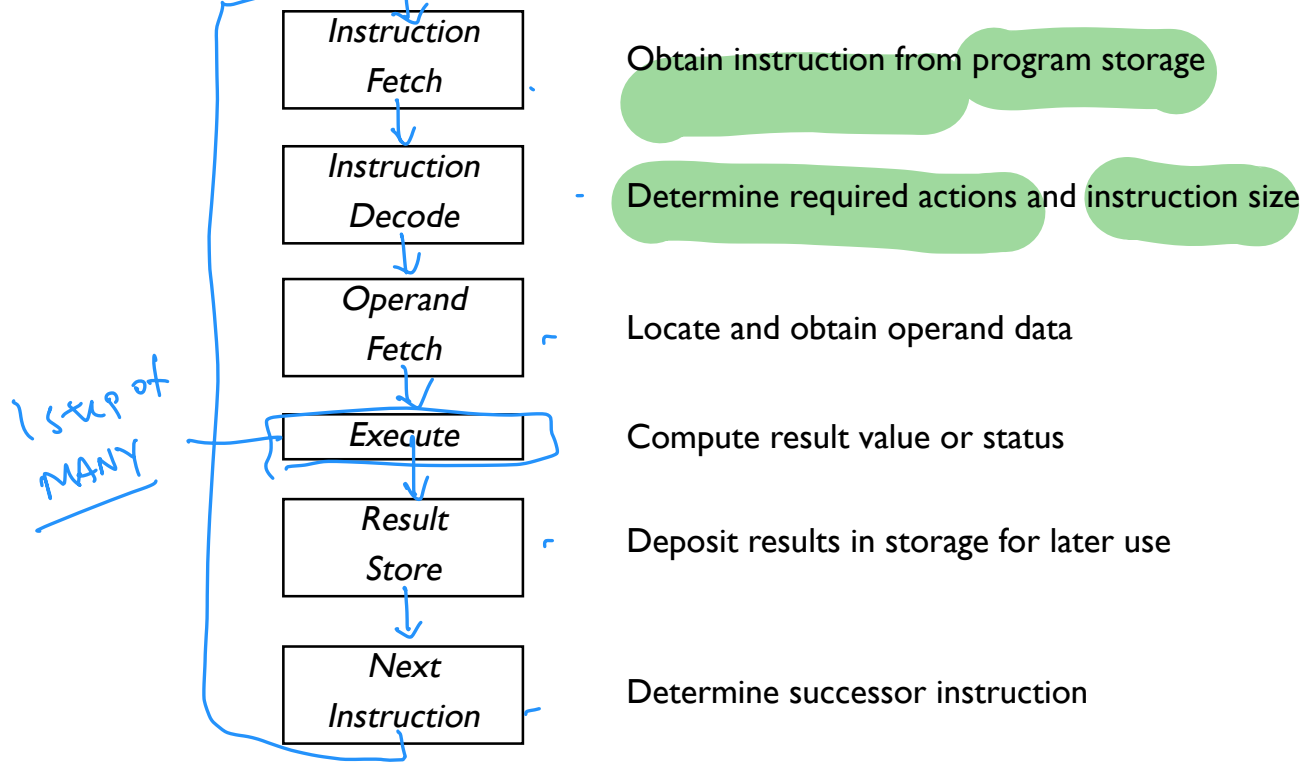


How does a computer execute instructions?

- Determined by the machine organization

How does a computer **execute instructions**?

- Determined by the **machine organization**



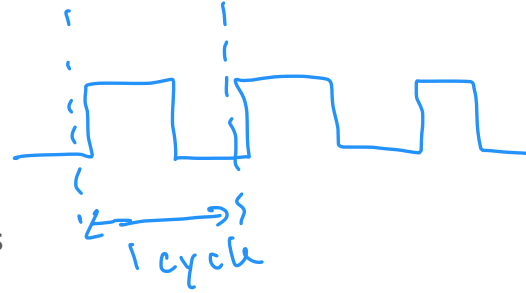
00110 = 6
↳ what operation
is encoded as 6?

Preview of machine organization concepts

- **Cycle**: the smallest unit of time in a processor

Preview of machine organization concepts

- **Cycle**: the smallest unit of time in a processor
 - Processors have a **clock**
 - Cycle = a "tick"
 - Cycle time a.k.a **clock speed** = how long a "tick" takes



How to determine the clock speed of my processor?

- Find out your chip's clock rate (a.k.a frequency)

$$1/s$$

- My Macbook Air has an M2 chip with a max clock rate of 3.49 GHz

- 1 GHz means 1 cycle per nanosecond

$$\frac{1}{ns}$$

$$\frac{1}{3.49 \text{ GHz}} = 0.287 \text{ ns} \quad \begin{array}{l} \text{cycle time} \\ \text{clock speed} \end{array}$$

How to determine the clock speed of my processor?

- The M5 chip has a max clock rate of 4.6 GHz. What is its cycle time?

- A. 4.6 ns
- B. 0.46 ns
- C. 217 ns
- D. 0.217 ns
- E. None of the above

Given a cycle...

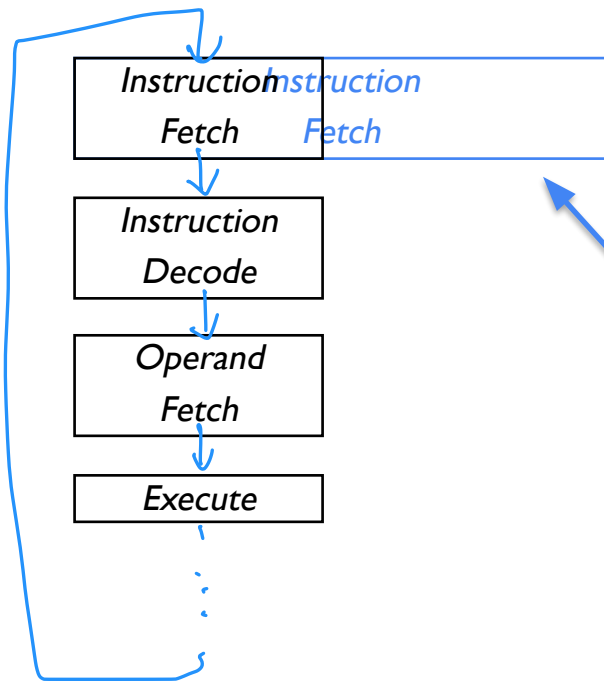
- Notice: Clock speed != performance !!!
- More important: How many instructions can execute per cycle?
 - Also known as the instruction rate
 - Depends on the architecture!

Preview: architecture design decisions

- Maximize parallelism

$2^4 = 16$
neg

register can hold 32 bits
32 bits
16-bit instr | 16-bit instr



Real-world example

ARM's Thumb instruction set is (mostly)
16-bit instructions on a 32-bit machine

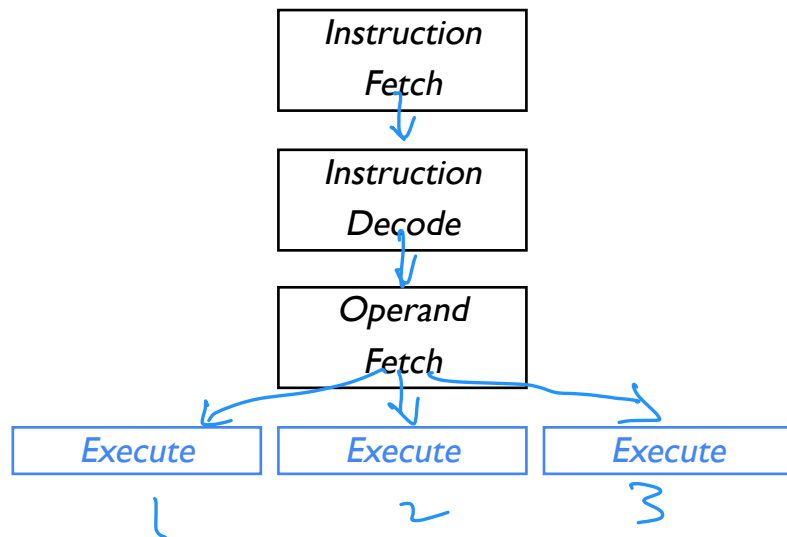
ISA design makes fetch “freely parallel”

What if we had 4-bit instr?

opcode neg

Preview: architecture design decisions

- A superscalar processor



Duplication is easy but expensive...

How to do parallelism well?

- Learn later in class

Preview: architecture design decisions

how much you can complete

5

increasing

- **Pipelining:** overlap parts of a large task to **increase throughput** w/o decreasing latency
- **Key insight:** The less work you do in one step, the faster each step can finish

↳ time

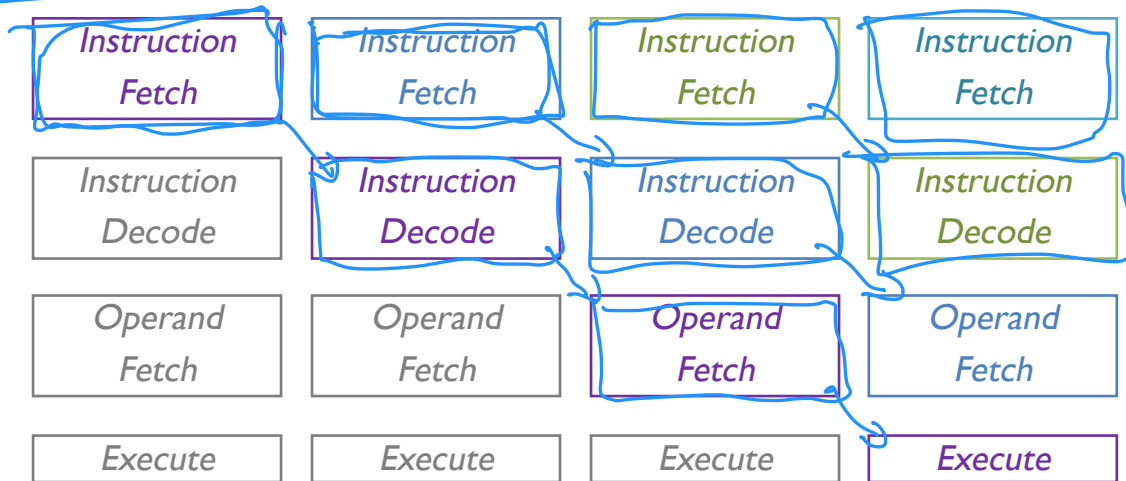
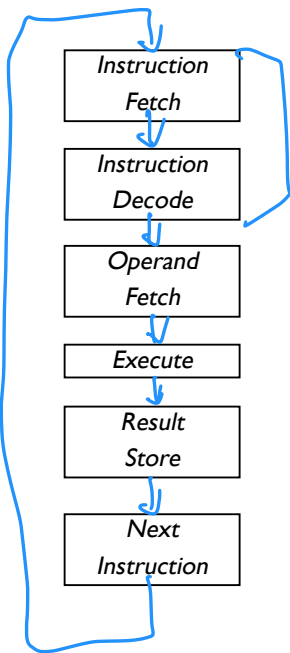
cycles:

1

2

3

4



How to design an ISA?

- Goal: Want an ISA that can be designed and executed efficiently in hardware

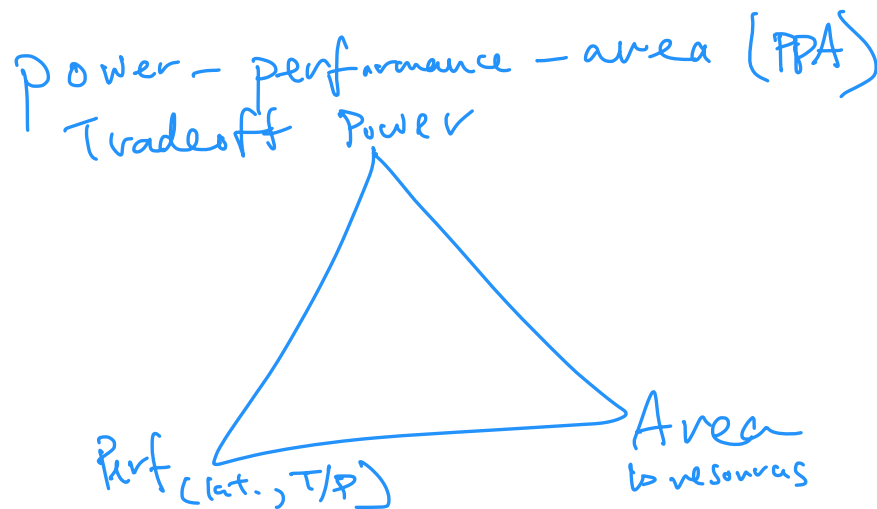
How to design an ISA?

- Goal: Want an ISA that can be designed and executed efficiently in hardware

- By efficiency, we usually focus on

- ↓ Latency — time — how long a task takes?
- ↑ Throughput # things / second
- ↓ Resource consumption
- ↓ Power consumption

There are always tradeoffs!



How to design an ISA?

We need

- **operations**: what an instruction does
- **operands**: specify what data is involved / impacted
- **instruction format**

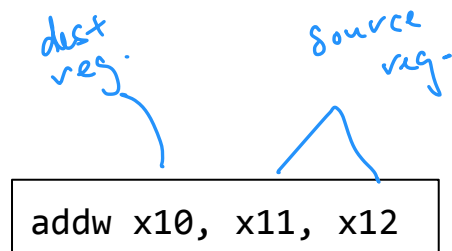
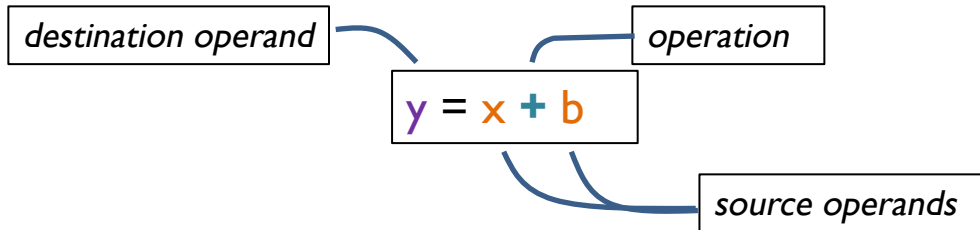
How to design an ISA?

We need

- **operations**: what an instruction does
- **operands**: specify what data is involved / impacted
- **instruction format**

dest reg. source reg.

addw x10, x11, x12

A diagram showing an assembly instruction 'addw x10, x11, x12' in a rectangular box. Above the box, there are two handwritten blue annotations. The first, 'dest reg.', has a blue arrow pointing to 'x10'. The second, 'source reg.', has a blue arrow pointing to 'x11'.

How to design an ISA?

- operations

- how many?
- which ones?

- operands

- how many?
- location
- types
- how to specify?

- instruction format

- size
- how many formats?

